

Session 1

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

1

INTRODUCTION

- About the instructor



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



2

COURSE PLAN SUMMARY

- First 3 weeks: Microcontrollers
 - Assessment in the Lab classes (presentation/discussion of proposed exercises)
- Last 2 week: Embedded Linux Systems
 - Assessment in lab classes (presentation/discussion of proposed exercises)



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



3

SUMMARY (FIRST WEEK)

- Review of microcontroller basics
 - Case study: Atmega 128



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



4

NOTE

- Public disclosure of these sessions is prohibited.
 - Public disclosure of any recordings or transcriptions is prohibited
 - The same applies to course materials



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



5

ATmega128

- CPU (single ALU, no FPU, RISC architecture) – launched around 2001/2002
 - AVR 8-bit architecture by Atmel, in 1997 (acquired by Microchip Technology in 2016)
 - 133 instructions
 - Up to 16 Million Instruction per second (MIPS) at 16 MHz
 - 32 auxiliary registers, STATUS register, program counter (PC), stack pointer (SP)
 - Memory
 - Program memory (128K bytes -> 64K x 16 bit words)
 - Internal Data Memory (4K x 8-bit)
 - Peripherals
 - 7 I/O ports (6 x 8 I/O pins + 1 x 5 I/O pins = 53 I/O pins)
 - 4 Timer/Counter
 - 2 USART (Universal Synchronous/Asynchronous Receiver/Transmitter)
 - ADC 10-bit (Analog to Digital Converter)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



6

ATmega128 vs Atmega328P (Arduino Uno)

- | | |
|--|--|
| <ul style="list-style-type: none"> ▪ Memory <ul style="list-style-type: none"> ▪ Program: 128 KiB ▪ Internal SRAM: 4 KiB <ul style="list-style-type: none"> ▪ Data memory map of 64 KiB ▪ EEPROM: 4 KiB ▪ Peripherals (main differences) <ul style="list-style-type: none"> ▪ 53 I/O lines ▪ 1 additional 16-bit Timer/Counter ▪ 2 additional 8-bit PWM channels ▪ 1 additional USART | <ul style="list-style-type: none"> ▪ Memory <ul style="list-style-type: none"> ▪ Program: 32 KiB ▪ Internal SRAM: 2 KiB <ul style="list-style-type: none"> ▪ Data memory map limited to internal addresses ▪ EEPROM: 1 KiB ▪ Peripherals (main differences) <ul style="list-style-type: none"> ▪ 23 I/O lines ▪ Other: <ul style="list-style-type: none"> ▪ Lower power consumption ▪ More compact |
|--|--|



ARDUINO UNO

- Prototyping board
- Low volume projects

SOFTWARE DEVELOPMENT FOR AVR

- We will mainly use the C programming language
 - C++ and assembly also officially supported
- Microchip Studio for AVR IDE (Windows)
 - <https://onlinedocs.microchip.com/oxy/GUID-ECD8A826-B1DA-44FC-BE0B-5A53418A47BD-en-US-12/index.html>
 - Alternatives for Linux:
 - MPLAB X (<https://www.microchip.com/en-us/tools-resources/develop/mplab-x-ide>)
 - Command line (avr-gcc)
 - Fedora: `dnf install avr-gcc avr-libc`
`dnf install avr-gcc-c++`
- AVR-Libc Documentation and repository
 - <https://avrdudes.github.io/avr-libc/>
 - <https://github.com/avrdudes/avr-libc>

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



9

QUESTION TO STUDENTS

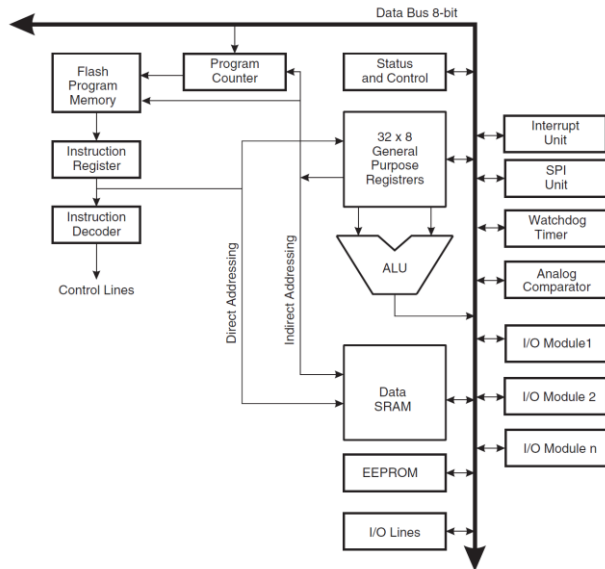
- Who will be using his/her own computer during the lab classes?
- Nevertheless, please install Microchip Studio for AVR “in background”, if possible.



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



10



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

ATMEGA128 ARCHITECTURE

- Harvard architecture

ntander

11

11

General
Purpose
Working
Registers

7	0	Addr.
R0		\$00
R1		\$01
R2		\$02
...		
R13		\$0D
R14		\$0E
R15		\$0F
R16		\$10
R17		\$11
...		
R26		\$1A
R27		\$1B
R28		\$1C
R29		\$1D
R30		\$1E
R31		\$1F

X-register Low Byte
X-register High Byte
Y-register Low Byte
Y-register High Byte
Z-register Low Byte
Z-register High Byte

MEMORY MAP -- GENERAL PURPOSE REGISTERS

- 8 bit registers
 - Hence the "8 bit architecture"
- Little-endian
- R26..R31 can be used for indirect addressing (16 bit addresses)

techmud

loop

engennara do porto

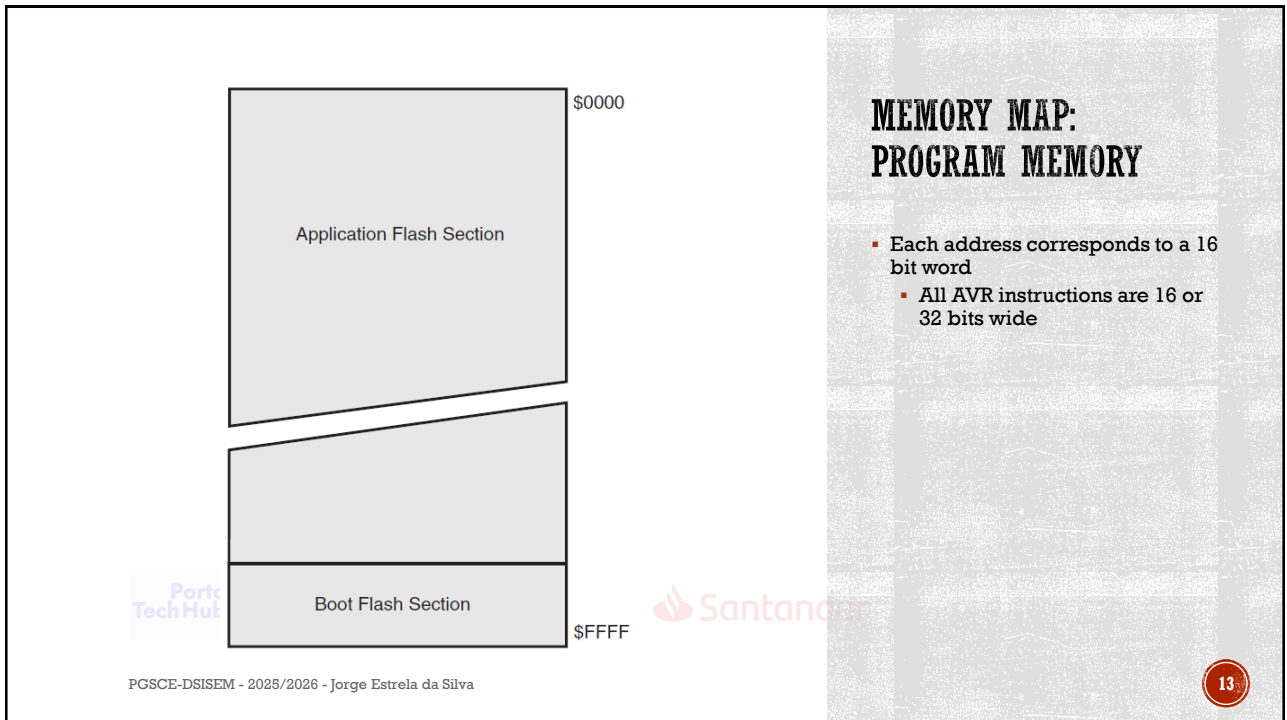
people

suntander

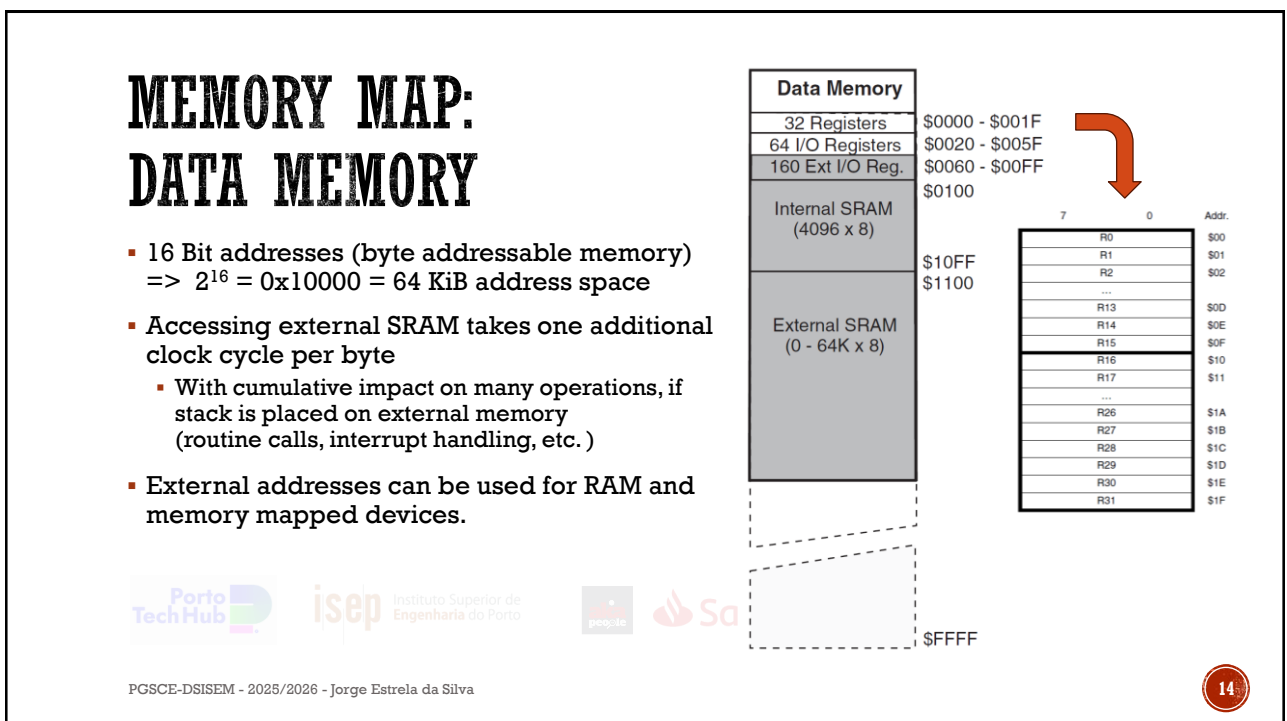
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

12

12



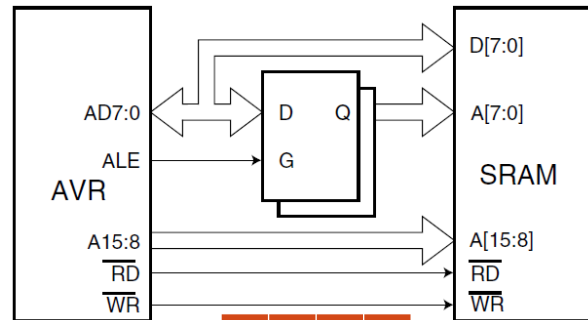
13



14

EXAMPLE: EXTERNAL SRAM

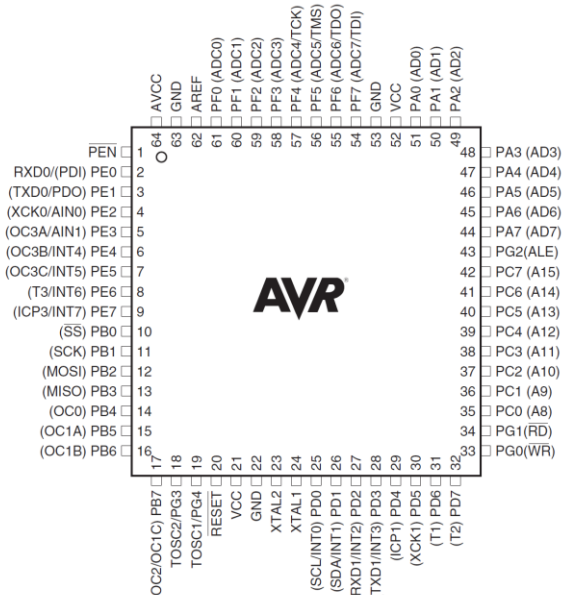
- PORTA is used for lines AD7:0
- PORTC is used for A15:8
- PG0 is used for WR
- PG1 is used for RD
- PG2 is used for ALE
- Gated D latch must support frequencies above 8 MHz (compliant with 74AHC series)



G	D	Q
0	X	→ Q
1	0	→ 0
1	1	→ 1

15

ATmega128 Pinout



16

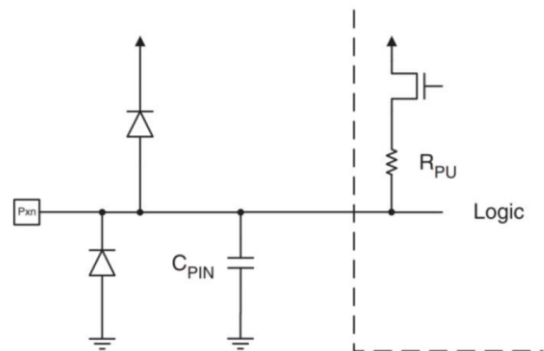
EEPROM DATA MEMORY

- 4 Kbytes
- Endurance of at least 100,000 write/erase cycles
- EEPROM accessed indirectly using registers in I/O space.

17

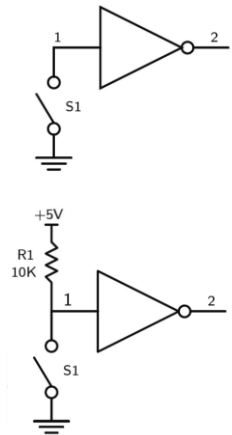
I/O PORTS

- 53 IO pins distributed by 7 “ports”
 - 6 x 8-pin ports (PORTA, ..., PORTF)
 - 1 x 5-pin port (PORTG)
- All pins are bidirectional with individually selectable pull-up resistors
- All I/O pins have protection diodes to both VCC and Ground (but don't push it...)
- All pins may be configured for digital Input or Output
 - Many of them can also be configured for alternate function (analog input, PWM, etc.)



18

PULL-UP RESISTOR (GENERAL IDEA)



Problem:

- If nothing is connected to pin 1, the entry value will be floating
- Any electrical noise can switch the output

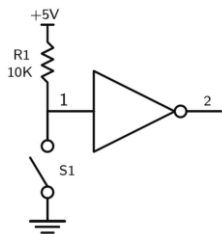
- The basic function of a pull-up resistor (R1) is to ensure that the input has always a "well-defined" value
- The pull-up resistor sets the input to the logical state 1, when nothing is connected to it

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

19

19

PULL-UP RESISTOR (GENERAL IDEA)



- When switch S1 is open (OFF), pin 1 is connected to Vcc through the resistor.
- When S1 is closed (ON), pin 1 has a direct GND connection which sets pin 1 logical level to zero.
 - A small current will flow, through R1, from VCC to GND => increased power consumption.

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

20

20

I/O PORT REGISTERS

Bit	7	6	5	4	3	2	1	0	
	PORTA7	PORTA6	PORTA5	PORTA4	PORTA3	PORTA2	PORTA1	PORTA0	PORTA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	DDA7	DDA6	DDA5	DDA4	DDA3	DDA2	DDA1	DDA0	DDRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	PINA7	PINA6	PINA5	PINA4	PINA3	PINA2	PINA1	PINA0	PINA
Read/Write	R	R	R	R	R	R	R	R	
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A	

- 3 registers for each I/O Port:
 - DDRx - PORTx Configuration (1 to set the pin as output, 0 for input)
 - PORTx
 - Output mode: the written value is driven to the pin
 - Input mode: used activate/deactivate the pull-up resistor (1 to activate)
 - Note: if PUD bit of the SFIOR register is set, internal pull-ups are disabled on all ports
 - PINx – Used to read PORTx pins' state (read only)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

21

21

PIN CONFIGURATION SUMMARY

DDxn	PORTxn	PUD (in SFIOR)	I/O	Pull-up	Comment
0	0	X	Input	No	Tri-state (Hi-Z)
0	1	0	Input	Yes	Pxn will source current if ext. pulled low.
0	1	1	Input	No	Tri-state (Hi-Z)
1	0	X	Output	No	Output Low (Sink)
1	1	X	Output	No	Output High (Source)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

22

22



C LANGUAGE EXAMPLE

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

25

VARIABLES IN C

- Basic data types
 - char (1 byte)
 - int (2 bytes !!!)
 - long (4 bytes)
 - long long (8 bytes)
 - float (4 bytes)
 - double (4 bytes !!!)
- Use <stdint.h> for integers:
 - uint8_t, int8_t, int16_t, uint16_t, etc.
- Variable declaration
 - uint16_t a;
 - uint8_t b, c, d;
 - int8_t vector[100]; // 4 KB of SRAM only...
- Immediate value assignment:
 - a = 15; // decimal
 - a = 0xF; // hexadecimal
 - c = 0b1111 //binary



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

26

26

BITWISE OPERATORS

- Bitwise OR (|)
 - Can be used to set bits
 - Example: `a |= (1 << 2)` to set bit 2 of a
- Bitwise AND (&)
 - Can be used to clear bits
 - Example: `a &= ~(1 << 2)` to clear bit 2 of a
- Bitwise XOR (^)
 - Can be used to toggle bits
 - Example: `a ^= (1 << 2)` to toggle bit 2 of a



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

27

27

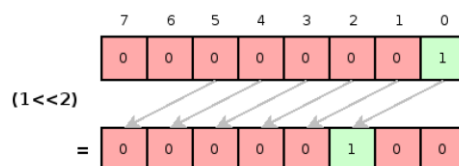
BITWISE OPERATIONS

- To manipulate a bit in a byte, it is used the bit mask

Example: 00000100 is the mask of bit 2

01000000 is the mask of bit 6

- To create the bit mask, It is used the operation shift left



- C Code in which the variable `a` represents the mask of bit 2

```
unsigned char a,b,c;
a = (1<<2); /* 00000100 */
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

28

28

LOGICAL OPERATORS

- Logical operators
 - `&&` AND
 - `||` OR
 - `!` NOT
- They should be used accordingly. For instance:
 - `uint8_t c = 1; //TRUE`
 - `!c //false`
 - `~c //true, because ~c equates to 0xFE (0b11111110)`



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

29

29

EXERCISE: IMPLEMENT A 2-INPUT LOGIC "AND" GATE USING PORTA

Symbol	Truth Table		
 2-input AND Gate	B	A	Q
	0	0	0
	0	1	0
	1	0	0
	1	1	1
Boolean Expression $Q = A.B$		Read as A AND B gives Q	



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

30

30

TESTING ON ARDUINO

- Microchip Studio can be used to program the Arduino Uno
 - Requires avrdude.exe, installed with Arduino IDE
 - Tools => External Tools
 - Command:
`C:\Users\jorge\AppData\Local\Arduino15\packages\arduino\tools\avrdude\8.0.0-arduino1\bin\avrdude.exe`
 - The prefix will depend on your configuration
 - Arguments:
`-C"C:\Users\jorge\AppData\Local\Arduino15\packages\arduino\tools\avrdude\8.0.0-arduino1\etc\avrdude.conf" -v -patmega328p -carduino -PCOM4 -b115200 -D -Uflash:w:"$(ProjectDir)Debug\$(TargetName).hex":I`
 - Check "Use Output window" to be able to see upload progress
 - Click "Add"
 - To upload the program, go again to "Tools", and you'll find a new menu option above "External Tools".

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

33

33

Session 2

Pós-Graduação em Sistemas Computacionais Embebidos (PGSCE) / SwitCH Embedded

Desenvolvimento de Sistemas Embebidos (PGSCE-DSISEM) / Embedded Systems Development - 2025/2026

Jorge Estrela da Silva

34



INTERRUPTS

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

35

INTERRUPTS

- The purpose of interrupts?



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



36

INTERRUPT SOURCES

- 8 External Interrupts
- 2 Timer/Counter Interrupts
 - 2 types of events
- 5 Timer/Counter1 Interrupts
- 2 Timer/Counter2 Interrupts
- 5 Timer/Counter3 Interrupts
- 3 USART0 Interrupts
- 3 USART1 Interrupts
- 1 ADC Interrupt
- 1 Analog Compare Interrupt
- 1 SPI Interrupt
- 1 Two Wire (~I2C) Interrupt
- 1 EEPROM Interrupt
- 1 Store Program Memory (SPM) Interrupt



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

37

37

Vector n°	Endereço	Fonte	Definição
1	0x0000	RESET	Operação de RESET
2	0x0002	INT0	Interrupção externa 0
3	0x0004	INT1	Interrupção externa 1
4	0x0006	INT2	Interrupção externa 2
5	0x0008	INT3	Interrupção externa 3
6	0x000A	INT4	Interrupção externa 4
7	0x000C	INT5	Interrupção externa 5
8	0x000E	INT6	Interrupção externa 6
9	0x0010	INT7	Interrupção externa 7
10	0x0012	TIMER2 COMP	T/C2 comparação
11	0x0014	TIMER2 OVIF	T/C2 overflow
12	0x0016	TIMER1 CAPT	T/C1 captura
13	0x0018	TIMER1 COMPA	T/C1 comparação A
14	0x001A	TIMER1 COMPB	T/C1 comparação B
15	0x001C	TIMER1 OVIF	T/C1 overflow
16	0x001E	TIMER0 COMP	T/C0 comparação
17	0x0020	TIMER0 OVIF	T/C0 overflow
18	0x0022	SPI, STC	SPI, transferência terminada
19	0x0024	USART0, RX	USART0, recepção terminada
20	0x0026	USART0, UDRE	USART0, registo de dados vazio
21	0x0028	USART0, TX	USART0, transmissão terminada
22	0x002A	ADC	ADC, conversão terminada
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Comparador analógico
25	0x0030	TIMER1 COMPC	T/C1 comparação C
26	0x0032	TIMER3 CAPT	T/C3 captura
27	0x0034	TIMER3 COMPA	T/C3 comparação A
28	0x0036	TIMER3 COMPB	T/C3 comparação B
29	0x0038	TIMER3 COMPC	T/C3 comparação C
30	0x003A	TIMER3 OVIF	T/C3 overflow
31	0x003C	USART1, RX	USART1, recepção terminada
32	0x003E	USART1, UDRE	USART1, registo de dados vazio
33	0x0040	USART1, TX	USART1, transmissão terminada
34	0x0042	TWI	Two Wire Interface
35	0x0044	SPM READY	Store Program Memory Ready

INTERRUPTS

- AVR architecture: pre-defined address for each interrupt vector
 - Typically, the code on those addresses will be a jump instruction to the address of the actual interrupt service routine (ISR)
 - In the case of simultaneous interrupts, the one with lower vector address is serviced first.
- Other architectures may differ. For instance, on the IA32 and x86_64 architectures, the address of the ISR must be stored on the Interrupt Descriptor Table (IDT), stored somewhere in the main memory (IDTR register stores the address of the IDT).

38

38

EXTERNAL INTERRUPTS

- It has 8 pins that can be configured individually to generate external interruptions.
 - INT3:0 -> PORTD 3:0 - Asynchronous external interrupts, independent of CLKIO
 - Pulse width must be greater than 50ns to generate an interrupt
 - INT7:4 -> PORTE 7:4 - Synchronous external interrupts, synchronized with CLKIO
 - Pulses must last longer than one clock period to generate an interrupt
- Each external interrupt can be configured to generate interruption in 3 different situations
 - Rising edge
 - Falling edge
 - Low level
- Whenever an external interrupt is activated, the interrupt will happen regardless of the pin configuration (Input or output).

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

39

39

INT3:0 CONFIGURATION

EICRA – External Interrupt Control Register A

Bit	7	6	5	4	3	2	1	0	
	ISC31	ISC30	ISC21	ISC20	ISC11	ISC10	ISC01	ISC00	EICRA
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ISCn1	ISCn0	Description
0	0	The low level of INTn generates an interrupt request.
0	1	Reserved
1	0	The falling edge of INTn generates asynchronously an interrupt request.
1	1	The rising edge of INTn generates asynchronously an interrupt request.

Note: 1. n = 3, 2, 1 or 0.

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

40

40

INT7:4 CONFIGURATION

EICRB – External Interrupt Control Register B

Bit	7	6	5	4	3	2	1	0	
	ISC71	ISC70	ISC61	ISC60	ISC51	ISC50	ISC41	ISC40	EICRB
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

ISCn1	ISCn0	Description
0	0	The low level of INTn generates an interrupt request.
0	1	Any logical change on INTn generates an interrupt request
1	0	The falling edge between two samples of INTn generates an interrupt request.
1	1	The rising edge between two samples of INTn generates an interrupt request.

Note: 1. n = 7, 6, 5 or 4.

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

41

41

INTERRUPT CONTROL

External Interrupt Mask

Bit	7	6	5	4	3	2	1	0	
	INT7	INT6	INT5	INT4	INT3	INT2	INT1	IINT0	EIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

42

42

INTERRUPT CONTROL

- Global Interrupt Enable bit on Status Register (SREG)

Bit	7	6	5	4	3	2	1	0	
	I	T	H	S	V	N	Z	C	SREG
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Automatically cleared when an interrupt occurs
- Automatically set when returning from the interrupt routine (RETI)
 - Meanwhile, all other interrupts are kept waiting!
 - Interrupt service routine (ISR) should be short
 - While possible to write 1 on GIE from the ISR to enable "nested" interrupts, that should be avoided

Porto TechHub

isep Instituto Superior de Engenharia do Porto

peccato

Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

43

43

EXTERNAL INTERRUPT FLAG REGISTER

Bit	7	6	5	4	3	2	1	0	
	INTF7	INTF6	INTF5	INTF4	INTF3	INTF2	INTF1	INTF0	EIFR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Flag cleared when the interrupt routine is executed
 - Can also be cleared by writing a 1 to it.

Porto TechHub

isep Instituto Superior de Engenharia do Porto

peccato

Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

44

44

EXTERNAL INTERRUPT CONFIGURATION SUMMARY

- Disable Interrupt on EIMSK
- Configure EICRA/EICRB
- Clear the flag on EIFR (by setting it to 1)
- Enable Interrupt on EIMSK
- Enable Interrupts globally on SREG (if not already enabled)
 - sei()/cli() on AVR C Lib
- Associations:
 - INT3:0 -> PORTD 3:0
 - INT7:4 -> PORTE 7:4



Instituto Superior de
Engenharia do Porto



Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

45

45

USING INTERRUPTS IN AVR LIB C

- Interrupt service routines declared with ISR() macro.
- Predefined names for each interrupt vector (XXXXXXX_vect)
 - #include <avr/interrupt.h>
 - For example, for **INT0**, the predefined symbol is **INT0_vect**:
 - ISR(INT0_vect) {
 -
 - }



Instituto Superior de
Engenharia do Porto



Santander

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

46

46

VOLATILE

- Whenever a function is not explicitly called (ex: **ISR(INT0_vect)**), the compiler can optimize access to variables so that a change in the interrupt routine may be ignored in the main line of program execution.
 - In order to avoid this problem, those variables should be declared as **volatile**

```
volatile int my_flag = 0;
ISR(INT0_vect) {
    my_flag = 1;
}
int main() {
    //...
    while(!my_flag);
}
```

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

47

47

USING INTERRUPTS IN C: EXERCISE

- Write a program that toggles the value of bit 0 of PORTB (PB0) whenever a falling edge is detected on bit 2 of PORTD (PD2).



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

48

48


```

#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint32_t counter = 0;

int main(void) {
    DDRB |= (1 << PB0); // Configure PB0 as output

    PORTD |= (1 << PD2); // Enable internal pull-up on PD2
    EICRA |= (1 << ISC21); // Falling edge => 10 on EICRA5:4
    EIMSK |= (1 << INT2); // Enable INT2 (PD2)
    sei(); // Enable interrupts globally

    while (1) {
        ++counter;
    }
}

ISR(INT2_vect) {
    PORTB ^= (1 << PB0); // Toggle PB0's state
}

```



Instituto Superior de
Engenharia do Porto



Santander



TIMER/COUNTER

DELAYS (BUSY WAIT)

- While useful for simple programs or for very short delays, they are not the most efficient solution in general.
 - Not energy efficient (prevents power-save mode)
 - Depends on the definition of F_CPU constant. If F_CPU does not match the actual operating speed, the timing will be wrong.
 - Interrupt service routines may add up for the total time
- `#define F_CPU 16000000UL //16 MHz oscillator`
- `#include <util/delay.h>`
- `void _delay_ms (double __ms)`
- `void _delay_us (double __us)`

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

51

51

ATMEGA128 TIMERS/COUNTERS

- 2 x 8 bit Timer/Counter
 - Timer/Counter 0
 - Timer/Counter 2
- 2 x 16 bit Timer/Counter
 - Timer/Counter 1
 - Timer/Counter 3

Porto TechHub isep Instituto Superior de Engenharia do Porto



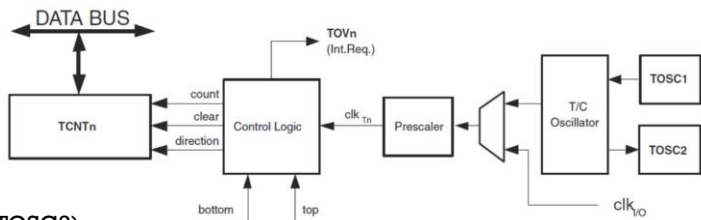
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

52

52

TIMER/COUNTER 0

- Clock source
 - Internal (CLKIO)
 - External Oscillator (TOSC1 and TOSC2)
 - PG3:4 – optimized for 32.768 kHz
- Prescaler 1, 8, 32, 64, 128, 256 or 1024
- TCNT0 – Counter register (8 bits)
- TOV0 – timer overflow flag (set to 1 when TCNT0 becomes 0)
 - Can be used as an interrupt source



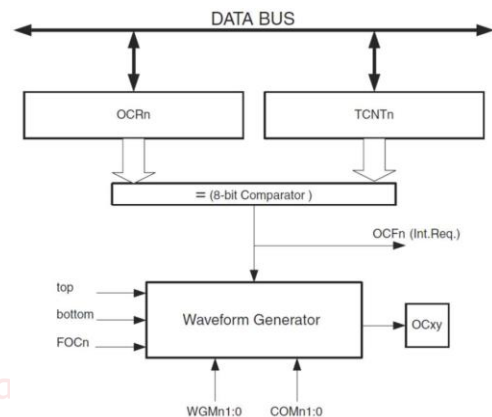
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

53

53

TIMER/COUNTER 0 – OUTPUT COMPARE UNIT

- Continuously compares the value of the counter (TCNT0) with the value of OCR0 (Output Compare Register 0)
- If equal sets flag OCF0 (Output Compare Flag0)
 - Can be used as an interrupt source (OCIE0=1)
- Depending on the settings, a signal may be generated on the OC0 Pin (PB4, for T/C0).

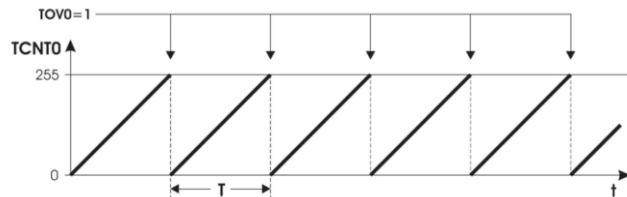


PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

54

54

T/C0 MODE 0 (NORMAL)



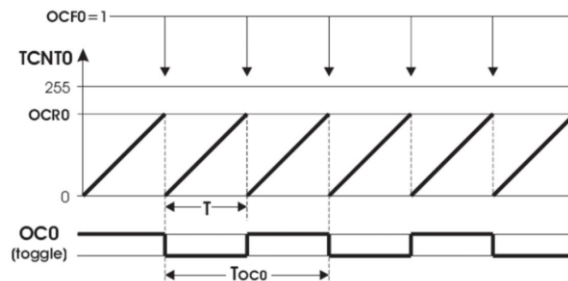
- The TCNT0 increments until it reaches the value 0xFF (TOP) restarting counting from 0x00 (BOTTOM).
- The TOV0 flag will be set in the same timer clock cycle as the TCNT0 becomes zero.
- TOV0 flag may generate an interrupt.
- $T = \text{Prescaler} / \text{CLKIO} \times 256$

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

55

55

T/C0 — MODE 2 (CTC) CLEAR TIMER ON COMPARE



- In CTC mode the counter is cleared to zero when the counter value (TCNT0) matches the OCR0
- The OCF0 is set in the clock cycle following the match.
 - The OCF0 flag may generate an interrupt
- OC0 pin can generate a waveform output (pin PB4)
- $T = \text{Prescaler} / \text{CLKIO} \times (\text{OCR0} + 1) \Leftrightarrow \text{OCR0} = T \times \text{CLKIO} / \text{Prescaler} - 1$
- The OVFO flag behaves as in the normal mode (but with no significant use)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

56

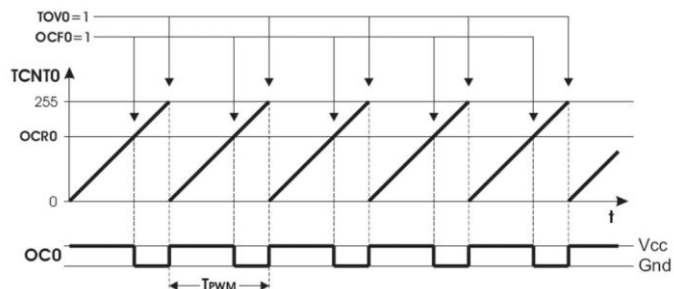
56

T/C0 – MODE 2 (CTC) – EXAMPLES

- Times in μs , assuming a 16 MHz clock

Prescaler	OCR0																
	0	0x0F	0x1F	0x2F	0x3F	0x4F	0x5F	0x6F	0x7F	0x8F	0x9F	0xAF	0xBF	0xCF	0xDF	0xEF	0xFF
1	0,0625	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
2	0,125	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30	32
4	0,25	4	8	12	16	20	24	28	32	36	40	44	48	52	56	60	64
8	0,5	8	16	24	32	40	48	56	64	72	80	88	96	104	112	120	128
32	2	32	64	96	128	160	192	224	256	288	320	352	384	416	448	480	512
64	4	64	128	192	256	320	384	448	512	576	640	704	768	832	896	960	1024
128	8	128	256	384	512	640	768	896	1024	1152	1280	1408	1536	1664	1792	1920	2048
256	16	256	512	768	1024	1280	1536	1792	2048	2304	2560	2816	3072	3328	3584	3840	4096
1024	64	1024	2048	3072	4096	5120	6144	7168	8192	9216	10240	11264	12288	13312	14336	15360	16384

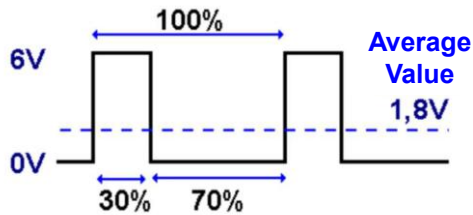
T/C0 MODE 3 (FAST PWM)



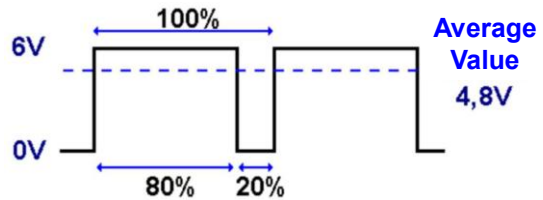
- The TCNT0 increments until it reaches the value 0xFF (TOP) starting from the 0x00 (BOTTOM).
- The OCF0 flag is set when the TCNT0 reaches the value OCR0
 - TOV0 flag is set when TCNT0 becomes 0x00.
- OCF0 and TOV0 flags may generate interrupt.
- OC0 pin can generate a waveform output.
- $T = \text{Prescaler}/\text{CLKIO} \times 256$

PULSE WIDTH MODULATION (PWM)

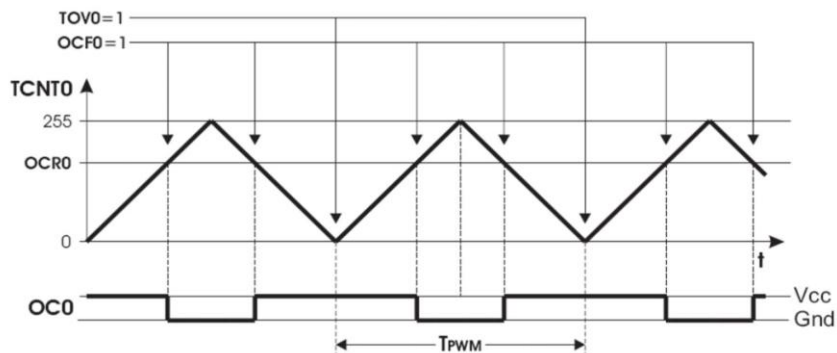
30% duty cycle



80% duty cycle



T/C0 – MODE 1 (PWM)



■ $T_{PWM} = \text{Prescaler} / \text{CLKIO} \times 510$

T/CO REGISTERS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

▪ Timer control Register (TCCR)

▪ Timer Counter Register (TCNT)

▪ Output Compare Register (OCR)

Bit	7	6	5	4	3	2	1	0	
	TCNT0[7:0]								TCNT0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

Bit	7	6	5	4	3	2	1	0	
	OCR0[7:0]								OCR0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

61

61

T/CO CONFIGURATIONS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

▪ WGM01:0 – Operation mode

Modo	WGM01	WGM00	Timer/Counter Modo de operação
0	0	0	Normal
1	0	1	PWM, Phase Correct
2	1	0	CTC
3	1	1	Fast PWM

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

62

62

T/CO CONFIGURATIONS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- COM01:0 – Compare Match Output mode
 - Non-PWM Mode:

COM01	COM00	Descrição
0	0	Normal port operation, OC0 disconnected
0	1	Toggle OC0 on compare match
1	0	Clear OC0 on compare match
1	1	Set OC0 on compare match

Por
Tech Hi

T/CO CONFIGURATIONS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- COM01:0 – Compare Match Output mode
 - Fast PWM Mode:

COM01	COM00	Descrição
0	0	Normal port operation, OC0 disconnected
0	1	Reserved
1	0	Clear OC0 on compare match, set OC0 at BOTTOM
1	1	Set OC0 on compare match, clear OC0 at BOTTOM

Te

T/C0 CONFIGURATIONS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

COM01:0 – Compare Match Output mode

Phase Correct PWM Mode:

COM01	COM00	Descrição
0	0	Normal port operation, OC0 disconnected
0	1	Reserved
1	0	Clear OC0 on compare match when up-counting. Set OC0 on compare match when downcounting
1	1	Set OC0 on compare match when up-counting. Clear OC0 on compare match when downcounting

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

65

65

T/C0 CONFIGURATIONS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

CS02:0 – Clock Select (Prescaler)

CS02	CS01	CS00	Descrição
0	0	0	No clock source (T/C stopped)
0	0	1	clk _{TOS} (No prescaling)
0	1	0	clk _{TOS} / 8 (From prescaler)
0	1	1	clk _{TOS} / 32 (From prescaler)
1	0	0	clk _{TOS} / 64 (From prescaler)
1	0	1	clk _{TOS} / 128 (From prescaler)
1	1	0	clk _{TOS} / 256 (From prescaler)
1	1	1	clk _{TOS} / 1024 (From prescaler)

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

66

66

T/C0 CONFIGURATIONS

Bit	7	6	5	4	3	2	1	0	
	FOC0	WGM00	COM01	COM00	WGM01	CS02	CS01	CS00	TCCR0
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- FOC0 – Force Output Compare (for non PWM modes)
 - Simulate a match. OCF0 flag not set, but OC0 pin is updated.
 - Should be left as 0 otherwise



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

67

67

T/C0 INTERRUPT SYSTEM

Bit	7	6	5	4	3	2	1	0	
	OCF2	TOV2	ICF1	OCF1A	OCF1B	TOV1	OCF0	TOV0	TIFR
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- Timer/Counter Interrupt Flag Register
 - OCF0 – Output Compare Flag0
 - TOV0 – Timer/Counter0 Overflow Flag



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

68

68

T/C0 INTERRUPT SYSTEM

Bit	7	6	5	4	3	2	1	0	
	OCIE2	TOIE2	TICIE1	OCIE1A	OCIE1B	TOIE1	OCIE0	TOIE0	TIMSK
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- **Timer/Counter Interrupt Mask Register**
 - OCIE0 – Timer/Counter0 Output Compare Match Interrupt Enable
 - TOIE0 – Timer/Counter0 Overflow Interrupt Enable



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

69

69

Vector nº	Endereço	Fonte	Definição
1	0x0000	RESET	Operação de RESET
2	0x0002	INT0	Interrupção externa 0
3	0x0004	INT1	Interrupção externa 1
4	0x0006	INT2	Interrupção externa 2
5	0x0008	INT3	Interrupção externa 3
6	0x000A	INT4	Interrupção externa 4
7	0x000C	INT5	Interrupção externa 5
8	0x000E	INT6	Interrupção externa 6
9	0x0010	INT7	Interrupção externa 7
10	0x0012	TIMER2 COMP	T/C2 comparação
11	0x0014	TIMER2 OVF	T/C2 overflow
12	0x0016	TIMER1 CAPT	T/C1 captura
13	0x0018	TIMER1 COMPA	T/C1 comparação A
14	0x001A	TIMER1 COMPB	T/C1 comparação B
15	0x001C	TIMER1 OVF	T/C1 overflow
16	0x001E	TIMER0 COMP	T/C0 comparação
17	0x0020	TIMER0 OVF	T/C0 overflow
18	0x0022	SPI, STC	SPI, transferência terminada
19	0x0024	USART0, RX	USART0, recepção terminada
20	0x0026	USART0, UDRE	USART0, registo de dados vazio
21	0x0028	USART0, TX	USART0, transmissão terminada
22	0x002A	ADC	ADC, conversão terminada
23	0x002C	EE READY	EEPROM ready
24	0x002E	ANALOG COMP	Comparador analógico
25	0x0030	TIMER1 COMPC	T/C1 comparação C
26	0x0032	TIMER3 CAPT	T/C3 captura
27	0x0034	TIMER3 COMPA	T/C3 comparação A
28	0x0036	TIMER3 COMPB	T/C3 comparação B
29	0x0038	TIMER3 COMPC	T/C3 comparação C
30	0x003A	TIMER3 OVF	T/C3 overflow
31	0x003C	USART1, RX	USART1, recepção terminada
32	0x003E	USART1, UDRE	USART1, registo de dados vazio
33	0x0040	USART1, TX	USART1, transmissão terminada
34	0x0042	TWI	Two Wire Interface
35	0x0044	SPM READY	Store Program Memory Ready

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

70

70

INTERRUPT VECTORS

EXERCISE

- Configure Timer1 for normal mode operation and enable interruption on overflow.
- The ISR must sample the value of a counter variable continuously incremented on the main function.
- Debug the program and inspect the variables value at each Timer0 overflow.
 - Test for different prescaler values.

```
int main(void) {
    while (1) {
        counter++;
    }
}

ISR(TIMER0_OVF_vect) {
    sample = counter;
    counter = 0;
}
```



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

71

71

EXERCISE

```
#include <avr/io.h>
#include <avr/interrupt.h>

volatile uint16_t counter;
volatile uint16_t sample;

int main(void) {
    // 1/8 prescaler
    // TCCR0 |= (1 << CS01);
    // 1/32 prescaler
    TCCR0 = (1 << CS01) | (1 << CS00);

    TIMSK |= (1 << TOIE0);
    sei();

    while (1) {
        cli();
        counter++;
        sei();
    }
}
```



Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

72

72

DISASSEMBLY: JUMP FROM PREDEFINED ADDRESS TO USER DEFINED ROUTINE

00000020 RJMP PC+0x0049

50: ISR(TIMER0_OVF_vect) {

00000069 PUSH R1 Push register on stack

0000006A PUSH R0 Push register on stack

0000006B IN R0,0x3F In from I/O location (**Status Register**)

0000006C PUSH R0 Push register on stack



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

73

73

DISASSEMBLY: NOT ALL C INSTRUCTIONS ARE ATOMIC!

32: counter++;

0000005E LDS R24,0x0100

Load direct from data space (0x0100)

00000060 LDS R25,0x0101

Load direct from data space (0x0101)

00000062 ADIW R24,0x01

Add immediate to word

00000063 STS 0x0101,R25

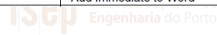
Store direct to data space (0x0101)

00000065 STS 0x0100,R24

Store direct to data space (0x0100)

Instruction Set Summary

Mnemonics	Operands	Description	Operation	Flags	#Clocks
ARITHMETIC AND LOGIC INSTRUCTIONS					
ADD	Rd, Rr	Add two Registers	$Rd \leftarrow Rd + Rr$	Z,C,N,V,H	1
ADC	Rd, Rr	Add with Carry two Registers	$Rd \leftarrow Rd + Rr + C$	Z,C,N,V,H	1
ADIW	RdI, K	Add Immediate to Word	$RdH:RdL \leftarrow RdH:RdL + K$	Z,C,N,V,S	2



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

74

74

POLLING VERSION (NO INTERRUPTS)

```
#include <avr/io.h>

volatile uint16_t counter;
volatile uint16_t sample;

int main(void) {
    // 1/8 prescaler
    // TCCR0 |= (1 << CS01);
    // 1/32 prescaler
    TCCR0 = (1 << CS01) | (1 << CS00);

    while (1) {
        counter++;
        if( TIFR & (1 << TOV0) ){
            TIFR |= (1 << TOV0); //clear flag by writing 1 to it
            sample = counter;
            counter = 0;
        }
    }
}
```



Instituto Superior de
Engenharia do Porto



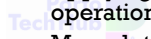
PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

75

75

TOSC1 AND TOSC2

- Asynchronous Status Register (ASSR)
 - Set bit 3 (AS0) to 1 to switch clock input to TOSC1
 - When AS0 is set, writes to TCNT0, OCR0, or TCCR0 take one TOSC cycle
 - Flags on ASSR can be used to control write operations on these registers
- When the value of AS0 is changed, TCNT0, OCR0 and TCCR0 might be corrupted
 - New values should be written
- External oscillator is not synchronized with CPU clock
 - Synchronization of the interrupt flags takes three processor cycles plus one timer cycle
- The oscillator (TOSC1+TOSC2) is optimized for use with a 32.768 kHz watch crystal.
 - Applying an external clock to the TOSC1 pin may result in incorrect Timer/Counter0 operation.
 - More details on the datasheet...



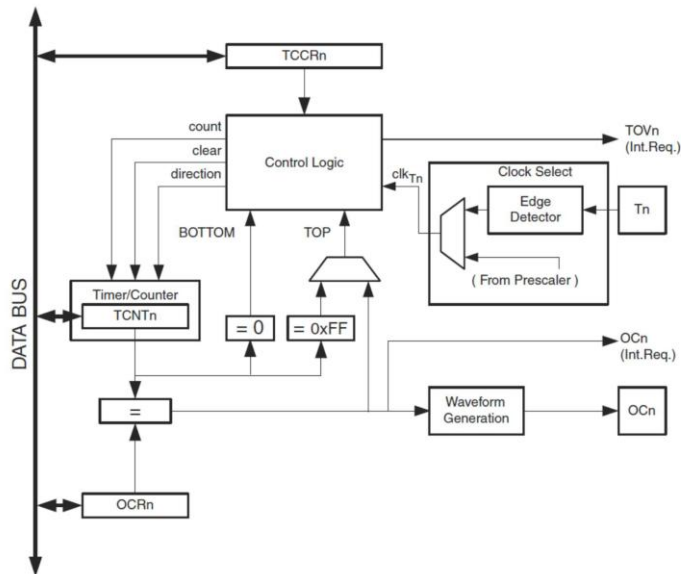
Instituto Superior de
Engenharia do Porto



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

76

76



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

TIMER/COUNTER2

- Similar to Timer/Counter 1
- External clock source, selectable in CS22:0 (T2)
 - Less prescaler options

index

77

77

T/C2 CONTROL REGISTER

Bit	7	6	5	4	3	2	1	0	
	FOC2	WGM20	COM21	COM20	WGM21	CS22	CS21	CS20	TCCR2
Read/Write	W	R/W	R/W	R/W	R/W	R/W	R/W	R/W	
Initial Value	0	0	0	0	0	0	0	0	

- CS22:0 – Clock Select

CS22	CS21	CS20	Description
0	0	0	No clock source (Timer/Counter stopped)
0	0	1	clkI/O/(No prescaling)
0	1	0	clkI/O/8 (From prescaler)
0	1	1	clkI/O/64 (From prescaler)
1	0	0	clkI/O/256 (From prescaler)
1	0	1	clkI/O/1024 (From prescaler)
1	1	0	External clock source on T2 pin. Clock on falling edge
1	1	1	External clock source on T2 pin. Clock on rising edge

PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

78

78



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva

79

ACKNOWLEDGEMENTS

- These slides are partially based on the notes from the **Microprocessors and Microcontrollers** course unit of the **Electrical and Computer Engineering** degree from **Instituto Superior de Engenharia do Porto**, with some of them being direct transcriptions of slides from that course unit.
- Several diagrams and tables from the ATmega128 datasheet are used on these slides:
 - <https://ww1.microchip.com/downloads/en/DeviceDoc/doc2467.pdf>



PGSCE-DSISEM - 2025/2026 - Jorge Estrela da Silva



80